
An Introduction to Reinforcement Learning in Interactive Systems

Modeling Sequential Decision Making in HCI

Thomas Langerak / Summer School on Computational Interaction 2026

thomas.langerak@aalto.fi

About Me

Thomas Langerak

Aalto University

Computational Behavior Lab

<https://www.thomaslangerak.nl>

thomas.langerak@aalto.fi

B.Sc. & M.Sc.
Design & HCI

TU/e



A!

Aalto-yliopisto
Aalto-universitetet
Aalto University

Ph.D.
CS (HCI)

ETH zürich

∞ Meta

Postdoc
HCI/RL

A!

Aalto-yliopisto
Aalto-universitetet
Aalto University

A!

Accompanying: Jupyter Notebook

The slides are accompanied by a Jupyter Notebook

During the lecture we will do some exercises in the notebook.

Please start it now, so that we can switch back and forth.

The notebook has a lot of text, during the class you can just ignore it.

Today

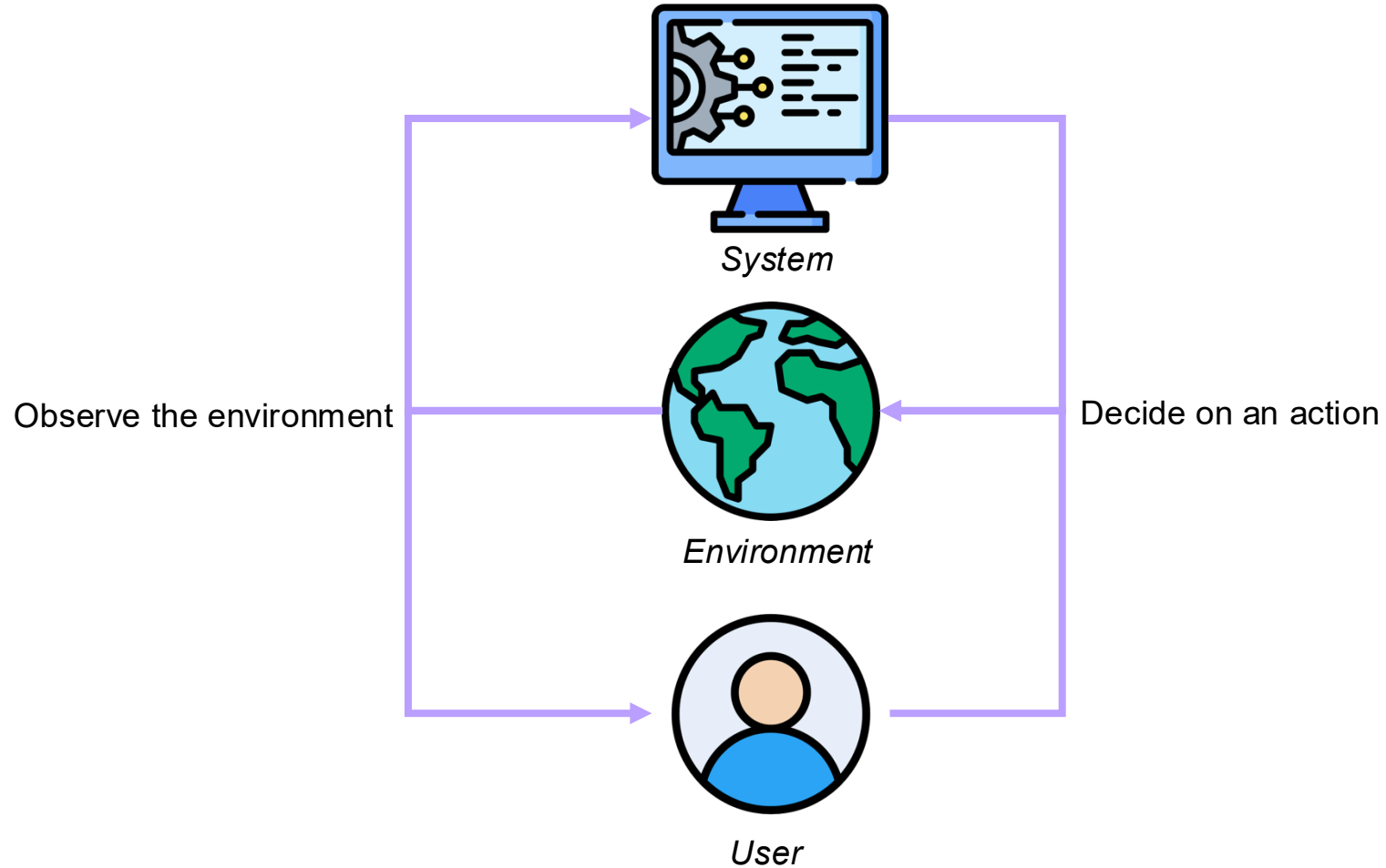
Learning Goals:

- Explain why HCI problems are sequential decision making problems.
- Able to formalize sequential decision making problems as a Markov Decision Process (MDP)
- Solve MDPs with Reinforcement Learning
- **In reality:** provide you the right terms to google for.

Agenda:

- Problem Definition
- Motivating RL
- Markov Decision Processes
- Reinforcement Learning

How to decide what to do, based on an observation?



A!

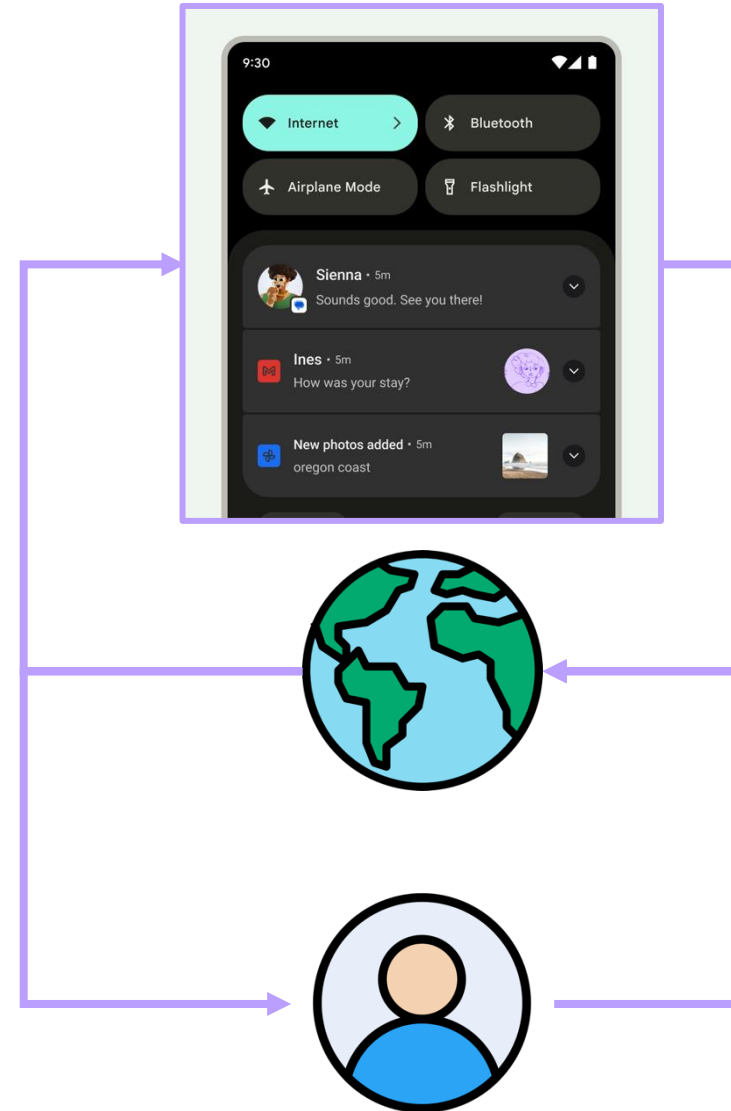
Assumption: There is something we want to achieve → There is a reward.

When to send a notification as a system?

Assume you are a phone.

- You receive multiple messages for the user, with different levels of importance and urgency.
- The user might be busy with important work.

- **Observe:** What might be useful to sense?
- **Act:** When do you notify the users of messages?
- **Reward:** What would you maximize for?



A!

Ex 1. Exploring the notification environment.

Jupyter Notebook has an environment that mimics the notification setting.

- At every time step there is a probability of a message coming in, with a random urgency and importance.
- The “payout” of notifying the user depends on the importance and how old the message is (important pays higher, urgent decays faster)
- The user has a concentration level, however it is not directly observable. We can only see activity and time of day.
- **At every time step you have to decide whether you notify the user or not.**

Ex 2. Implementing three naïve policies.

- 2a. Random: equally likely to push or do nothing
- 2b. Heuristic: Think of a simple rule (e.g., push every n steps)
- 2c. Myopic: maximize the immediate reward

All of these methods seemed pretty bad...

Method	Issue
Random	Random is never really optimal
Heuristic	Writing comprehensive rules is hard and error prone.
Myopic	An immediate payout is not always ideal.
Data-driven	Impossible to gather data.

Why is this so hard?

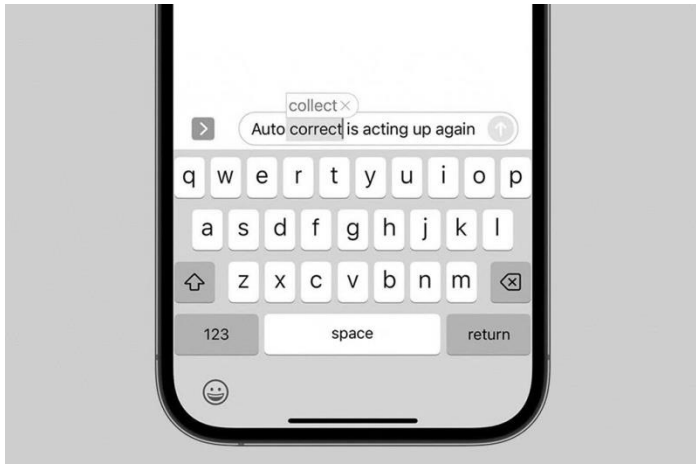
It is a **sequential decision making** problem.

- **Sequentially:** Multiple decisions need to be taken in sequence. Each action changes the world and subsequent decisions.
 - **Delayed Rewards:** Not every decision is guaranteed a reward, the high pay-off might only come after multiple correct decisions.
 - **Stochasticity:** Even if you take the same decision in the same state of the world, the outcome might be different
 - **Bounded:** Your actions cannot directly make the world as you wish.
- **Partial Observability:** You cannot observe everything (e.g., inside a user's head)

Sequential decision making is about making the **right choice**, **given the information** you have to **maximize a future payout**.

- **Given Information:**
We have some (incomplete) knowledge that we base our action on.
- **Right Choice:**
There is a set of actions to pick from, and some are better than others.
- **Maximize Future Payout:**
We optimize for some future states that rewards us.

Systems should make sequential decisions under partial observability in stochastic settings with a delayed reward.



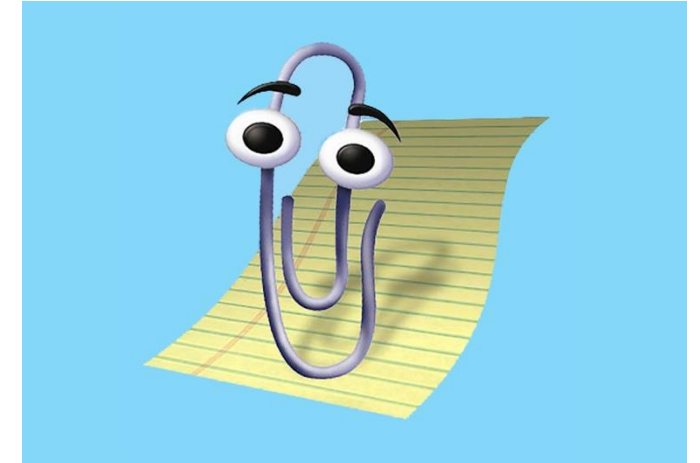
Interfaces

E.g., when to correct a word based on interaction history



Robotics

Where to move based on spatial context



Virtual Assistants

When to intervene based on user context and actions.

Sequential decision making is about making the **right choice**, **given the information** you have to **maximize a future payout**.

- **Given Information: States**
We have some (incomplete) knowledge that we base our action on.
- **Right Choice: Actions**
There is a set of actions to pick from.
- **Maximize Future Payout: Rewards**
We optimize for some future states that rewards us.

Markov Decision Process

- Models sequential decision making under uncertainty
- Captures the interaction between an agent and an environment over discrete steps
- Formal framework for reinforcement learning and stochastic control
- Assumes the Markov property: futures states depend only on the current state and action.

Markov Decision Processes

- **States:** An environmental representation (e.g., pixels, location, ...)
- **Actions:** The choices an agent can take (e.g., up/down, force applied, ...)
- **Transition:** The probability of transitioning to a new state, given the previous state and action.
- **Reward:** The utility of arriving in a new state
- **Discount Factor:** Rewards in the future are less important than rewards now.

$$\{S, A, T, R, \gamma\}$$

Partial Observable Markov Decision Processes

- **States:** An environmental representation (e.g., pixels, location, ...)
- **Actions:** The choices an agent can take (e.g., up/down, force applied, ...)
- **Transition:** The probability of transitioning to a new state, given the previous state and action.
- **Reward:** The utility of arriving in a new state
- **Discount Factor:** Rewards in the future are less important than rewards now.
- **Observations:** The set of possible observations
- **Observation Probabilities:** the probability of making a specific observation given a state.

$$\{S, A, T, R, \gamma, \Omega, O\}$$

Connection between (PO)MDP and RL

(PO)MDP: A prescriptive model. A description of the world with all that we know. An underlying formalism for RL

RL: A framework on how the agent can learn optimal behavior in an unknown or partially known MDP.

Ex 3. What is the POMDP of a notification system?

States:

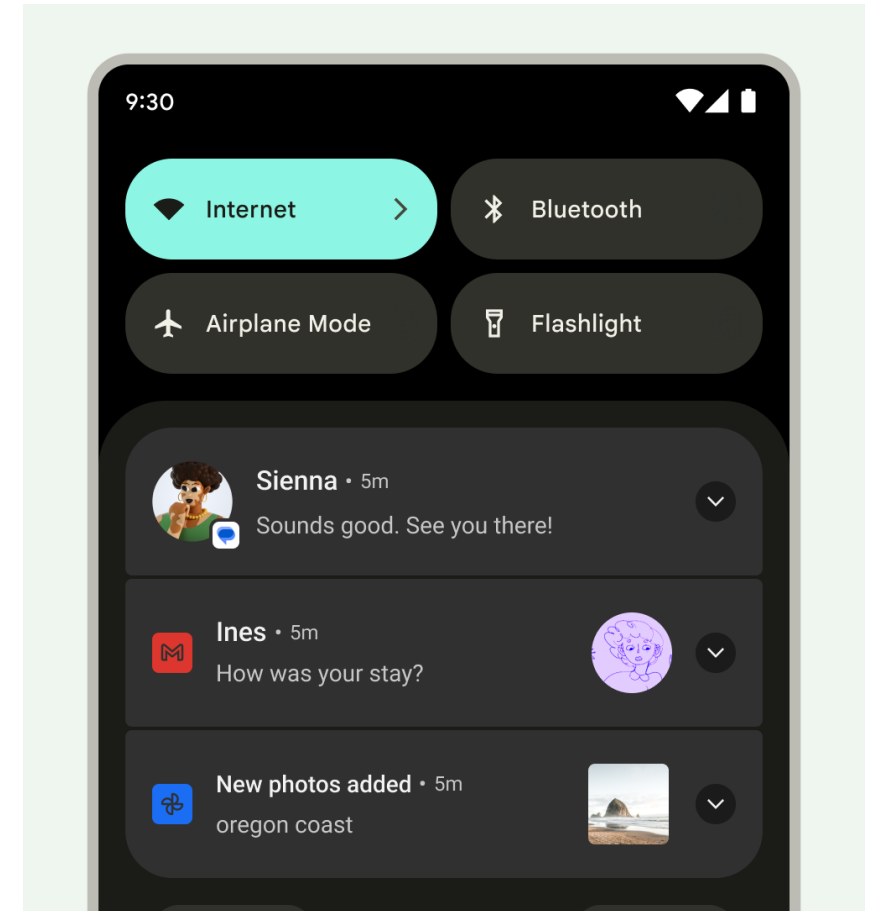
Actions:

Transition:

Reward:

Observations:

Observation Probabilities:



A!

Evaluating States

If we have this representation of the environment, a natural question to ask is:

How good is it to be in a specific state?

The Bellman Equation gives us the answer

A!

Policy and Value Function

We describe our strategy of which actions to take with a policy $\pi : S \rightarrow A$

Important concept: **Value** function $V_\pi : S \rightarrow \mathbb{R}$ of a policy π .

V_π expresses the expected cumulative reward (return) for each state that we achieve when following the policy π

Bellman equation for v_π

$$G_t \doteq \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The **return** is the sum of discounted rewards

$$v_\pi(s) \doteq E_\pi[G_t | S_t = s]$$

The **value** is the expected return of following a policy given a starting state

It's Recursive!

$$= \sum_a \underbrace{\pi(a|s)} \sum_{s'} \sum_r \underbrace{p(s', r | s, a)} [r + \gamma \underbrace{E_\pi[G_{t+1} | S_{t+1} = s']}]$$

Expanding the expectation

Policy:
Probability of an action given a state

Transition Matrix:
Probability of a state and reward, given a state and action

The immediate reward r plus the discounted expected future return from next state s' , following policy π .

A!

Bellman equation for v_π

$$v_\pi(s) = E_\pi[r + \gamma v_\pi(s') | S_t = s]$$

The value of a state (s) following a policy (π), is its expected immediate reward (r) plus the discounted (γ) value of the next state (s')

Intuitively: how good a situation you are in depends on what you gain right now, plus how good the situation you move into is. The discount γ reflects that future gains are worth less than immediate ones.

A!

Dynamic Programming

- Collection of algorithms that can compute optimal policies given a perfect model of the world (MDP).
- Can be applied to continuous space, we focus on discrete.
- Limited utility, great theoretical importance.
 - All RL methods have a similar goal, with less computation and no perfect model assumption.

A!

Multiple ideas, we will focus on Value Iteration

Value Iteration – Model-Based Planning

- Assumes full knowledge of the environment (MDP).
- Uses the Bellman optimality equation.
- Produces optimal value function and policy.

Value Iteration - Bellman Optimality Update Step

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [r(s, a, s') + \gamma V_k(s')]$$

- Intuitively: Refining the estimate of how good each state is by using the **best you action from that state onward**, based **on the current understanding** of the future.
- Over iterations, this pushes the value estimates to converge to the **optimal values**.

Value Iteration - Algorithm

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that
$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$$

Ex 4. Run the algorithm by hand

0	0	0	END 100
0	0	LAVA -100	LAVA -100
START 0	0	0	0

- Goal: Find optimal path to goal
- States: Grid fields
Start state blue field
Terminal states trap/goal
- Actions: Move left/right/up/down
- Reward: 100 upon reaching the goal
-100 landing in the trap
-5 for each step taken
- Other info: $\gamma = 1$
Initial states initialized to 0
Deterministic transitions

A!

$$V_{k+1}(s) = \max_a \{r(s, a, s') + \gamma V_k(s')\}$$

Solution (step cost = -5, $\gamma = 1$)

0	0	0	END 100
0	0	LAVA -100	LAVA -100
START 0	0	0	0

- Goal: Find optimal path to goal
- States: Grid fields
Start state blue field
Terminal states lava/goal
- Actions: Move left/right/up/down
- Reward: 100 upon reaching the goal
-100 landing in the lava
-5 for each step taken
- Other info: $\gamma = 1$
Initial states initialized to 0
Deterministic transitions

A!

$$V_{k+1}(s) = \max_a \{r(s, a, s') + \gamma V_k(s')\}$$

Solution (step cost = -5, $\gamma = 1$)

-5	-5	95	END 100
-5	-5	LAVA -100	LAVA -100
START -5	-5	-5	-5

- Goal: Find optimal path to goal
- States: Grid fields
Start state blue field
Terminal states trap/goal
- Actions: Move left/right/up/down
- Reward: 100 upon reaching the goal
-100 landing in the trap
-5 for each step taken
- Other info: $\gamma = 1$
Initial states initialized to 0
Deterministic transitions

A!

$$V_{k+1}(s) = \max_a \{r(s, a, s') + \gamma V_k(s')\}$$

Solution (step cost = -5, $\gamma = 1$)

-10	90	95	END 100
-10	-10	LAVA -100	LAVA -100
START -10	-10	-10	-10

- Goal: Find optimal path to goal
- States: Grid fields
Start state blue field
Terminal states trap/goal
- Actions: Move left/right/up/down
- Reward: 100 upon reaching the goal
-100 landing in the trap
-5 for each step taken
- Other info: $\gamma = 1$
Initial states initialized to 0
Deterministic transitions

A!

$$V_{k+1}(s) = \max_a \{r(s, a, s') + \gamma V_k(s')\}$$

Solution (step cost = -5, $\gamma = 1$)

85	90	95	END 100
-15	85	LAVA -100	LAVA -100
START -15	-15	-15	-15

- Goal:** Find optimal path to goal
- States:** Grid fields
Start state blue field
Terminal states trap/goal
- Actions:** Move left/right/up/down
- Reward:** 100 upon reaching the goal
-100 landing in the trap
-5 for each step taken
- Other info:** $\gamma = 1$
Initial states initialized to 0
Deterministic transitions

A!

$$V_{k+1}(s) = \max_a \{r(s, a, s') + \gamma V_k(s')\}$$

Solution (step cost = -5, $\gamma = 1$)

85	90	95	END 100
80	85	LAVA -100	LAVA -100
START -20	80	-20	-20

- Goal: Find optimal path to goal
- States: Grid fields
Start state blue field
Terminal states trap/goal
- Actions: Move left/right/up/down
- Reward: 100 upon reaching the goal
-100 landing in the trap
-5 for each step taken
- Other info: $\gamma = 1$
Initial states initialized to 0
Deterministic transitions

A!

$$V_{k+1}(s) = \max_a \{r(s, a, s') + \gamma V_k(s')\}$$

Solution (step cost = -5, $\gamma = 1$)

85	90	95	END 100
80	85	LAVA -100	LAVA -100
START 75	80	75	-25

- Goal:** Find optimal path to goal
- States:** Grid fields
Start state blue field
Terminal states trap/goal
- Actions:** Move left/right/up/down
- Reward:** 100 upon reaching the goal
-100 landing in the trap
-5 for each step taken
- Other info:** $\gamma = 1$
Initial states initialized to 0
Deterministic transitions

A!

$$V_{k+1}(s) = \max_a \{r(s, a, s') + \gamma V_k(s')\}$$

Solution (step cost = -5, $\gamma = 1$)

85	90	95	END 100
80	85	LAVA -100	LAVA -100
START 75	80	75	70

- Goal: Find optimal path to goal
- States: Grid fields
Start state blue field
Terminal states trap/goal
- Actions: Move left/right/up/down
- Reward: 100 upon reaching the goal
-100 landing in the trap
-5 for each step taken
- Other info: $\gamma = 1$
Initial states initialized to 0
Deterministic transitions

A!

$$V_{k+1}(s) = \max_a \{r(s, a, s') + \gamma V_k(s')\}$$

Ex 5: Let's implement in the notebook!

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

| $\Delta \leftarrow 0$

| Loop for each $s \in \mathcal{S}$:

| | $v \leftarrow V(s)$

| | $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

| | $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$$

Implement This!

A!

Pros and Cons of Dynamic Programming

Pros:

- Value iteration is guaranteed to converge in finite number of iterations
- Find optimal value and policy

Cons:

- Need to know the transition probability matrix
- Need to iterate over the whole state space (very expensive)
- Requires memory proportional to the size of the state space

The Era of Experience

Humans learn from what we directly experience, and not every single state in the world.

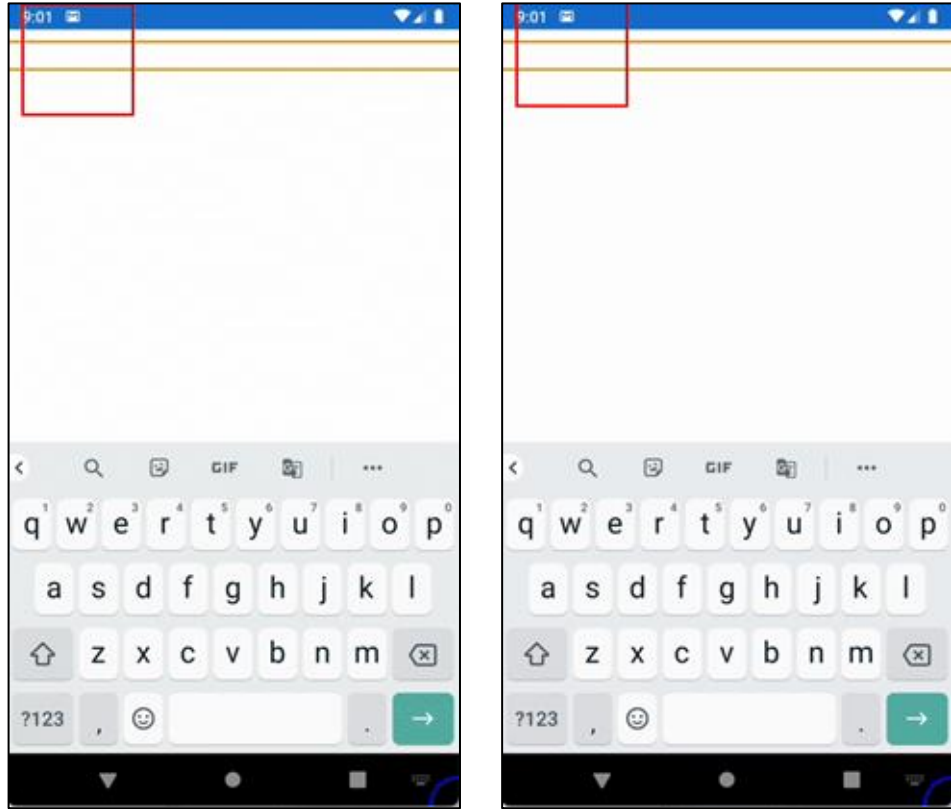
Agent's should be able to do the same.

Reinforcement Learning is the modeling and learning of sequential decision making.

Reinforcement learning is learning what to do, how to map situations to actions, so as to maximize a numerical reward signal. [1]

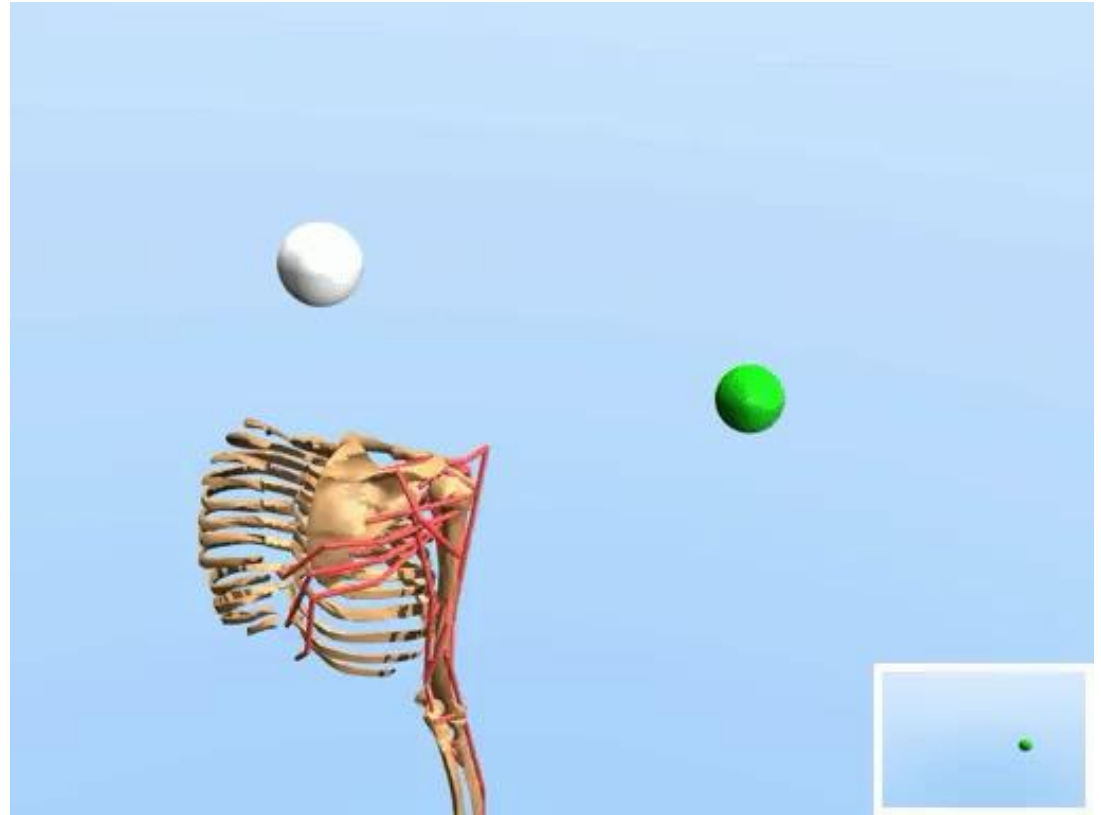
- **Learning from experience:** trial-and-error search

RL in HCI: Simulating Humans



Shi, D., Zhu, Y., Jokinen, J. P., Acharya, A., Putkonen, A., Zhai, S., & Oulasvirta, A. (2024, May). CRTypist: Simulating touchscreen typing behavior via computational rationality. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (pp. 1-17).

A!



Moon, H. S., Liao, Y. C., Li, C., Lee, B., & Oulasvirta, A. (2024, May). Real-time 3d target inference via biomechanical simulation. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (pp. 1-18).

RL in HCI: Adaptive Systems

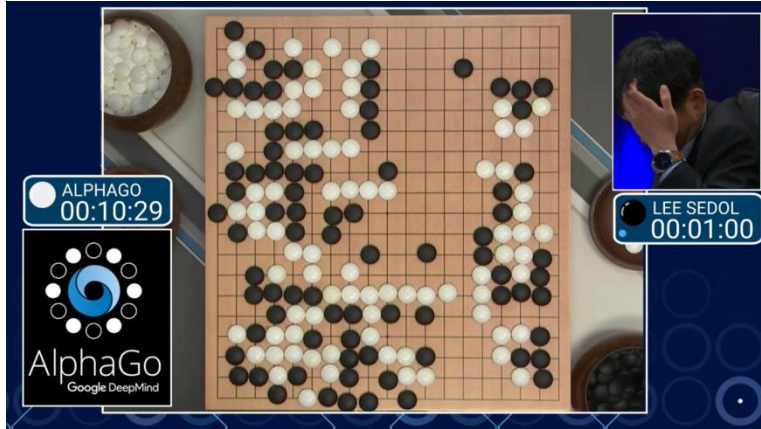


Given:
Interface
User State

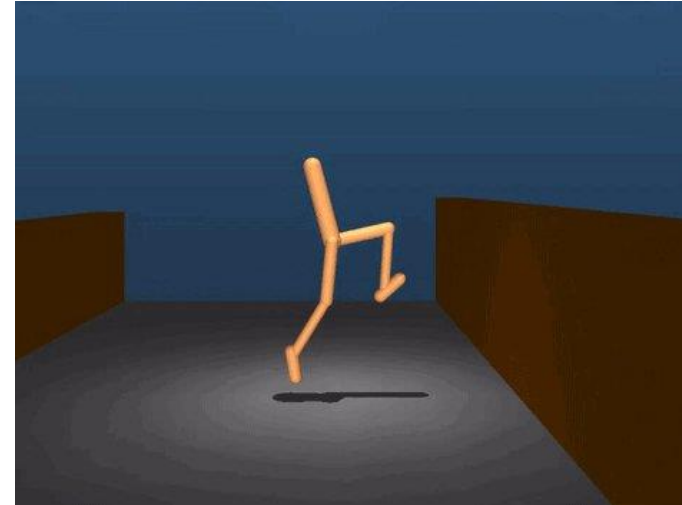
Decide:
Item-to-Menu Assignment

To maximize:
User performance

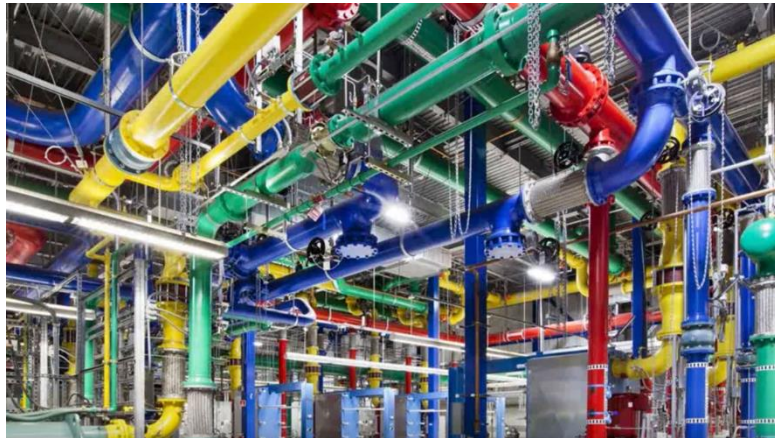
HCI can learn from the successes of other fields



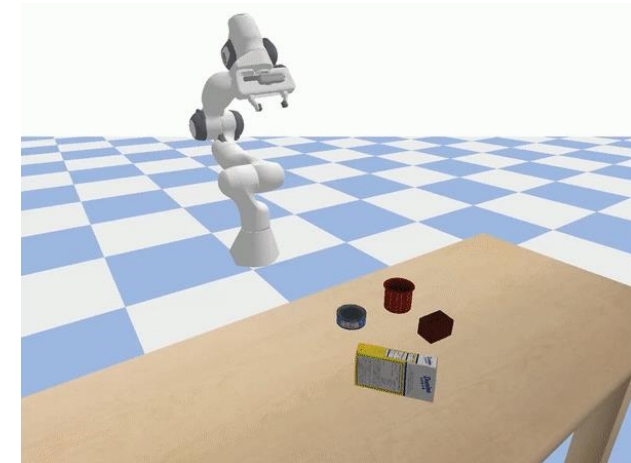
Games



Robot Control



Logistics and Operations



Robot Control + Computer Vision

V-function vs. Q-function

Q-value function

- Explicitly takes actions into account

$$v_{\pi}(s) \doteq E_{\pi}[G_t | S_t = s]$$

$$q_{\pi}(s, a) \doteq E_{\pi}[G_t | S_t = s, A_t = a]$$

- Helps with problems when the transition function of the environment is not available
- When action space is large, this may be problematic

A!

Problem!

We now need a policy, as it is not given to us and we cannot compute the whole state-space (as in Value Iteration).

What do we do?

A!

Learning from Experience: Exploration vs Exploitation

What policy can we use to interaction with the environment?

exploration: selecting random actions to visit unknown states

exploitation: selecting actions that are optimal given our current knowledge

What could be a good trade-off?

ϵ -greedy policy

In each state, with small probability ϵ choose randomly, else choose greedily

Is this ideal?

- Given its simplicity and generality, it works well in practice
- More sophisticated approaches exist (importance sampling)
- Decreasing ϵ can improve it (more exploration in the beginning)

Q-Learning

For each step with action a from s to s' that we take with our policy we compute the difference to our current estimate and update our value function:

$$a_t \sim \pi_{\epsilon}(s)$$

Target

$$\delta_t = r(s_t, a_t, s_{t+1}) + \lambda \max_{a'} [Q_k(s_{t+1}, a')] - Q_k(s_t, a_t)$$

Error

$$Q_{k+1}(s_t, a_t) = Q_k(s_t, a_t) + \alpha \cdot \delta$$

A!

Q-learning learns the value of the **greedy** policy (optimal), even while following an exploratory one.

Q-Learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

Ex 6: Let's implement in the notebook!

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

Initialize S

Loop for each step of episode:

Choose A from S using policy derived from Q (e.g., ε -greedy)

Take action A observe R, S'

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

$S \leftarrow S'$

until S is terminal

Implement This!

A!

Q-learning is generally more popular than SARSA, and has more follow-up methods

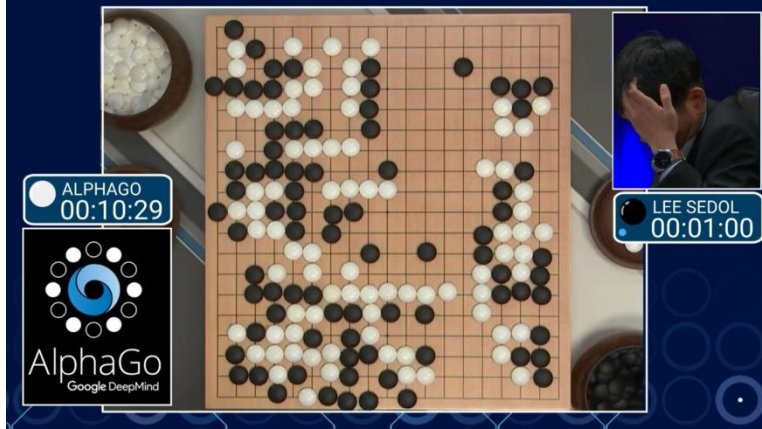
Pros and Cons of Temporal Differences (Q-learning)

Pros:

- Unbiased estimate
- Do not need to know system dynamics

Cons:

- High variance
- Exploration/Exploitation dilemma



Game of Go
 10^{170} States



Atari 256x256 grayscale
 image

10^{16000} States

A!

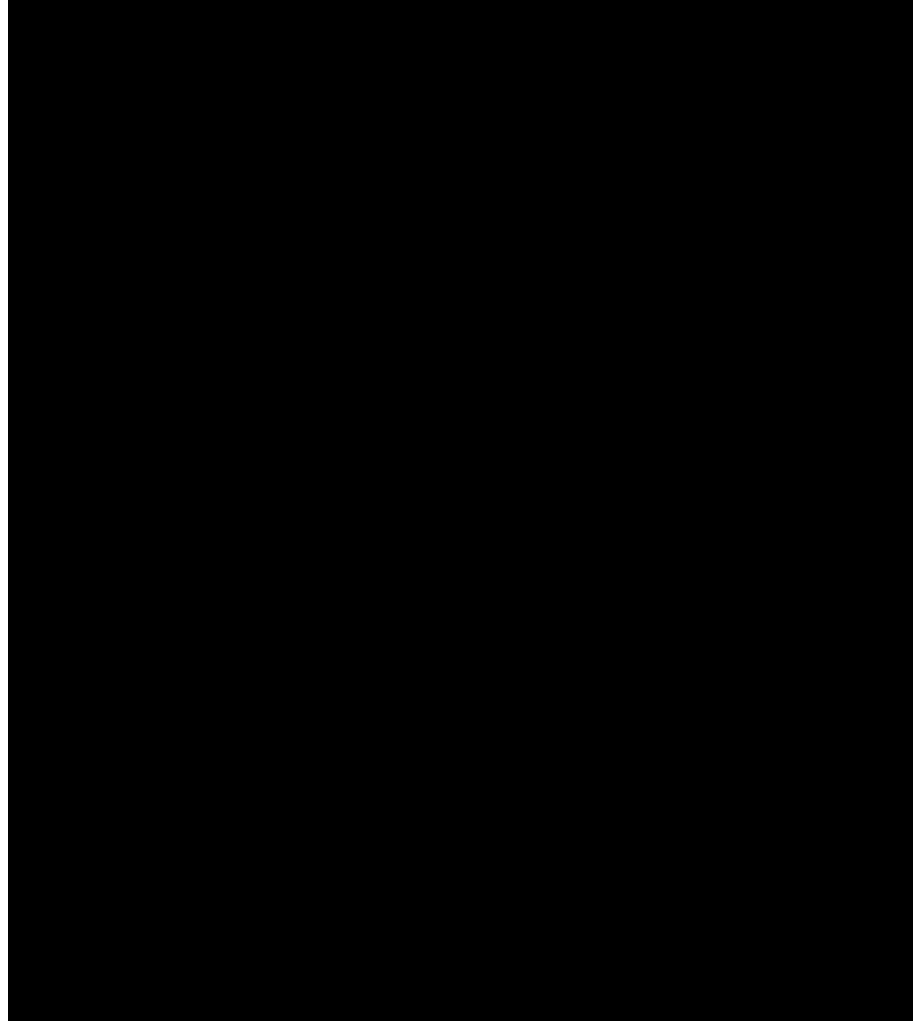
[Mnih et. al, 2013]

The Limits of Temporal Difference Methods

- **Problem:** TD methods (like Q-learning) require a tabular representation of value functions.
- **Scalability issue:** Doesn't scale to high-dimensional or continuous state spaces (e.g., raw images, rich UIs, motor control).
- **No generalization:** Each state-action pair is learned independently; experience in one state doesn't help in similar ones.
- **Example:** Modeling every possible visual layout or hand position is intractable.

So while TD learning can model toy tasks well, it breaks down when state spaces explode: as they do in real HCI scenarios.

Deep RL has been very successful



A!

Deep Reinforcement Learning

Everything we discussed so far, is just a **function**

$$\pi: S_t \rightarrow A_t$$

$$v_\pi: S_t \rightarrow V(S_t)$$

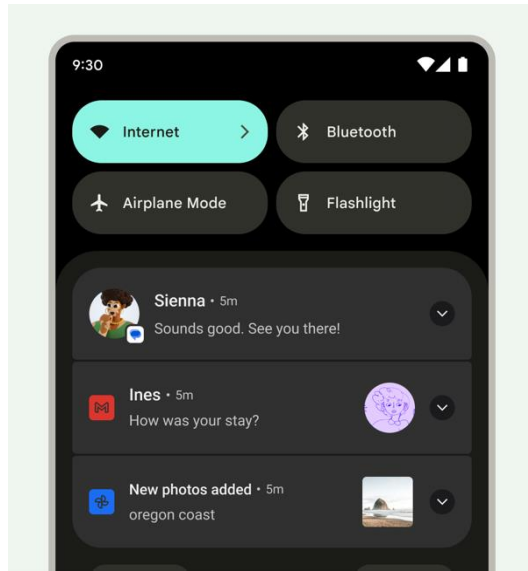
$$q_\pi: S_t, A_t \rightarrow Q(S_t, A_t)$$

Deep Reinforcement Learning

- Use neural networks as function approximator:
 - Parameterize V , Q , π , or all as a neural network.
- Use gradient based methods to update the policy

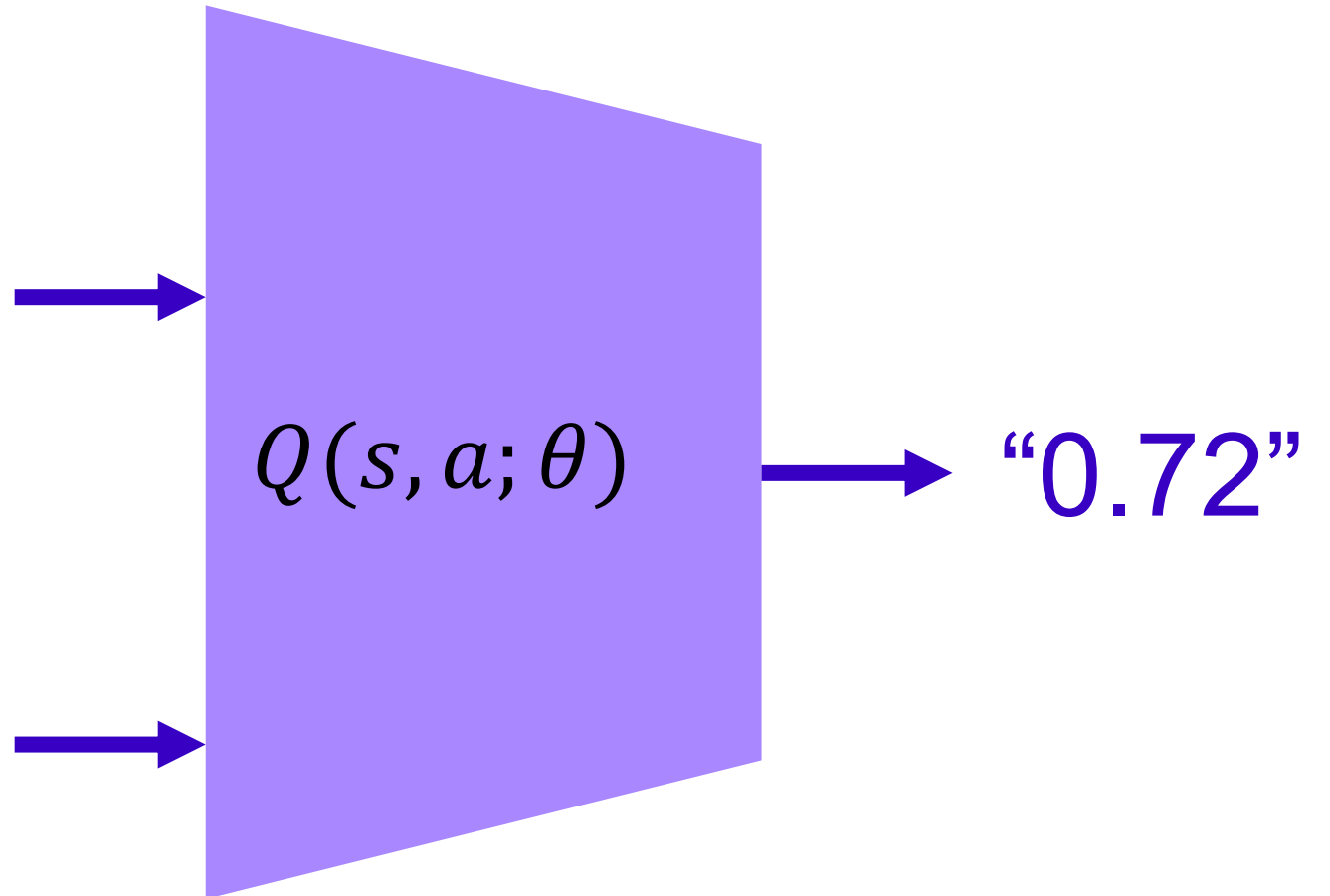
Neural Networks in Deep RL

S



a “Push notifications”

A!



Deep Q-Networks

Q-Learning:

$$\delta_t = r(s, a, s') + \lambda \max_{a'} [Q_k(s', a') - Q_k(s, a)]$$

Previously, we assumed a separate Q-value estimate for each state-action pair (s, a) .

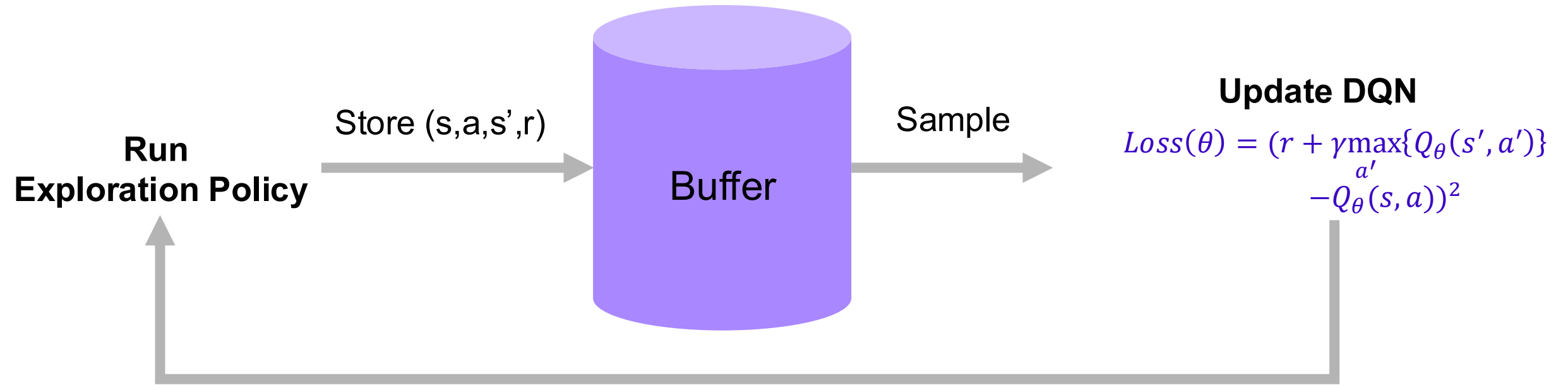
We can use a **neural network** to learn the mapping between state-action pairs (s, a) and their values.

The Q-learning updates reduce to **stochastic gradient descent** on:

$$Loss(\theta) = (r + \gamma \max_{a'} \{Q_\theta(s', a')\} - Q_\theta(s, a))^2$$

A!

Deep Q-Networks



A!

Exercise 7: Let's evaluate DQN

Actually, a lot of the algorithms are already implemented in packages.
We, like most researchers, will be using Stable-Baselines3.

Does it our perform Tabular Q-learning? Why? Why not?
What about value iteration?

A!

Today

- **Introduced MDP:** Describing an environment in terms of states, actions, rewards, transition, and discount.
- **Value Iteration:** Compute the value of a state, assuming full knowledge of the MDP
- **Learning from Experience:** First, what if we do not have full knowledge. Second, how can we learn planning
- **Deep Reinforcement Learning:** Extend to more complex environment via Neural Networks

Outlook

- Scratched the surface of RL. There is a lot of different flavors.
 - IRL: What if the reward is unknown
 - Behavioral Cloning/Imitation Learning/Offline RL: What if we have data we want a policy to mimic
- We only looked at single-agent settings. What about multi-agent?



Kiitos
aalto.fi

References

- [1] Sutton, R. S., & Barto, A. G. (1998). *Reinforcement learning: An introduction* (Vol. 1, No. 1, pp. 9-11). Cambridge: MIT press.
- [2] Marr, David. *Vision: A computational investigation into the human representation and processing of visual information*. MIT press, 2010.